PAGE :
DATE : 8,10,26

2. Particle swarm optimization.

Pseudocode.

```
initializing particles
For each particle:
    initialize position (x) - current solution
    initialize velocity (v) - determine how particle moves
    calculate fitness
    pbest = x

gbest = best pbest

For generation in generations:
    For each particle:
        r1 = random (0,1)
        r2 = random (0,1)
        v = w*v + c1*r1*(pbest - x) +
            c2*r2*(gbest - x)   (personal experience
        x = x + v
        calculate fitness (x)
        if fitness (x) > fitness (pbest):
            pbest = x

        if fitness (x) > fitness (gbest)
            gbest = x

print (gbest)
```

Robotics

PAGE :
DATE : 8, 10, 25

IEEE paper on Robotics:

Target Seeking Behaviour of an Intelligent Mobile Robot using Advanced PSO. (IEEE conference paper, 2013)

Paper summary

Goal: Developing two different motion planners for an autonomous mobile robot, inspired by particle swarm optimization, to generate feasible paths in an unknown environment.

Method: Each planner has its own fitness function, modelled from the robot's sensory information.

Results: Both planners produced collision-free paths and reached the destination within the workspace, and the paper compares the two planners.

Use:- Robot doesn't need a map in advance. It uses live sensor data and PSO to choose each next move.

Code summary:-

1. Set up: The robot starts at (1,1) the target is at target (9,9) & there are 6 circular obstacles
2. Each step: The robot starts at inital position (1,1) and creates a swarm of 25 particles, each one a candidate move within a 0.4m step.

PAGE :
DATE : / /

3. PSO loop: The particles run for 15 iterations, updating velocity from their own best and the swarm's best, and the robot then moves to the best candidate.
4. Fitness function:
   - Distance to the target (lower is better)
   - obstacle penalty, applied only to obstacles within the 1.5m sensor range.
   - A very large penalty if the move lands inside an obstacle.
5. Two planners:
   - Planner 1 uses distance plus obstacle penalty.
   - Planner 2 adds turning penalty for smoother paths & a revisit penalty.
6. stopping: The robot stops when it is within 0.3 m of the target or after 300 steps.
7. output: A table with reached or not, path length, steps, and collisions for each planner, plus two path plots.

Output:

Planner	Reached	length(m)	steps	collisions
1	0	8.460	300	0
2	1	14.914	41	0

when obs = np.array([
[3.0, 3.0, 1.0],
[5.0, 6.0, 1.2],
[7.5, 4.0, 1.0],
[4.0, 8.5, 0.8],
[8.0, 8.0, 0.6],
[2.0, 6.0, 0.7],])

completed coding

COI 8/10/26.